

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO BÀI TẬP LỚN
LẬP TRÌNH MẠNG

**Đề tài: Ironfront Reborn - Trò chơi bắn súng nhiều người chơi
với tầng mạng TCP/UDP tự xây dựng**

Lớp: D23CTPM01-B

Nhóm bài tập lớn: 10

Sinh viên thực hiện: Nguyễn Thành Trung

Mã sinh viên: B23DCCN861

Báo cáo kiến trúc và giao diện hệ thống

Phần cá nhân: *Phân hệ client Unity*

Hà Nội - 2026

Mục lục

I. NỘI DUNG NHÓM	3
1. Mô tả kiến trúc chung của hệ thống	3
1.1. Bài toán và phạm vi	3
1.2. Kiến trúc ba tiến trình	3
1.3. Phân chia TCP và UDP theo đặc tính dữ liệu	5
1.4. Kiến trúc phân lớp hệ thống	5
1.5. Kiến trúc triển khai	6
1.6. Phân công theo phân hệ	7
2. Mô tả thiết kế chung của hệ thống	7
2.1. Mô hình thẩm quyền	7
2.2. Luồng nghiệp vụ đầu - cuối	8
2.3. Các kỹ thuật netcode cốt lõi	9
2.4. Thiết kế giao thức	9
2.5. Tham số thời gian	10
2.6. Phân loại đối tượng đồng bộ	11
2.7. Bảo mật ở mức phù hợp phạm vi	11
2.8. Các quyết định kiến trúc đã chốt	11
2.9. Quy trình phát triển và bảo đảm chất lượng	11
II. NỘI DUNG CÁ NHÂN - Phân hệ client Unity.	13
3. Biểu đồ và giao diện nội dung cá nhân thực hiện	13
3.1. Điểm nối netcode trong mã gameplay	13
3.2. Thứ tự thực thi một khung hình client	14
3.3. Máy trạng thái luồng trò chơi	14
3.4. Giao diện sảnh chờ	15
3.5. Giao diện trong trận và overlay chẩn đoán	17
3.6. Luồng dự đoán và hoà giải phía client	18
4. Luồng xử lý và giải pháp đã ứng dụng	18
4.1. Luồng đăng nhập từ giao diện	18
4.2. Luồng vào phòng và kết nối game server	20
4.3. Luồng dự đoán và hoà giải chuyển động người chơi cục bộ	21
4.4. Luồng hiển thị người chơi và bot ở xa	22
4.5. Luồng sự kiện chiến đấu, mục tiêu và HUD	23
4.6. Luồng mất kết nối và kết thúc trận	23
4.7. Bảng ánh xạ hình vẽ với chức năng cá nhân thực hiện	24

I. NỘI DUNG NHÓM

1. Mô tả kiến trúc chung của hệ thống

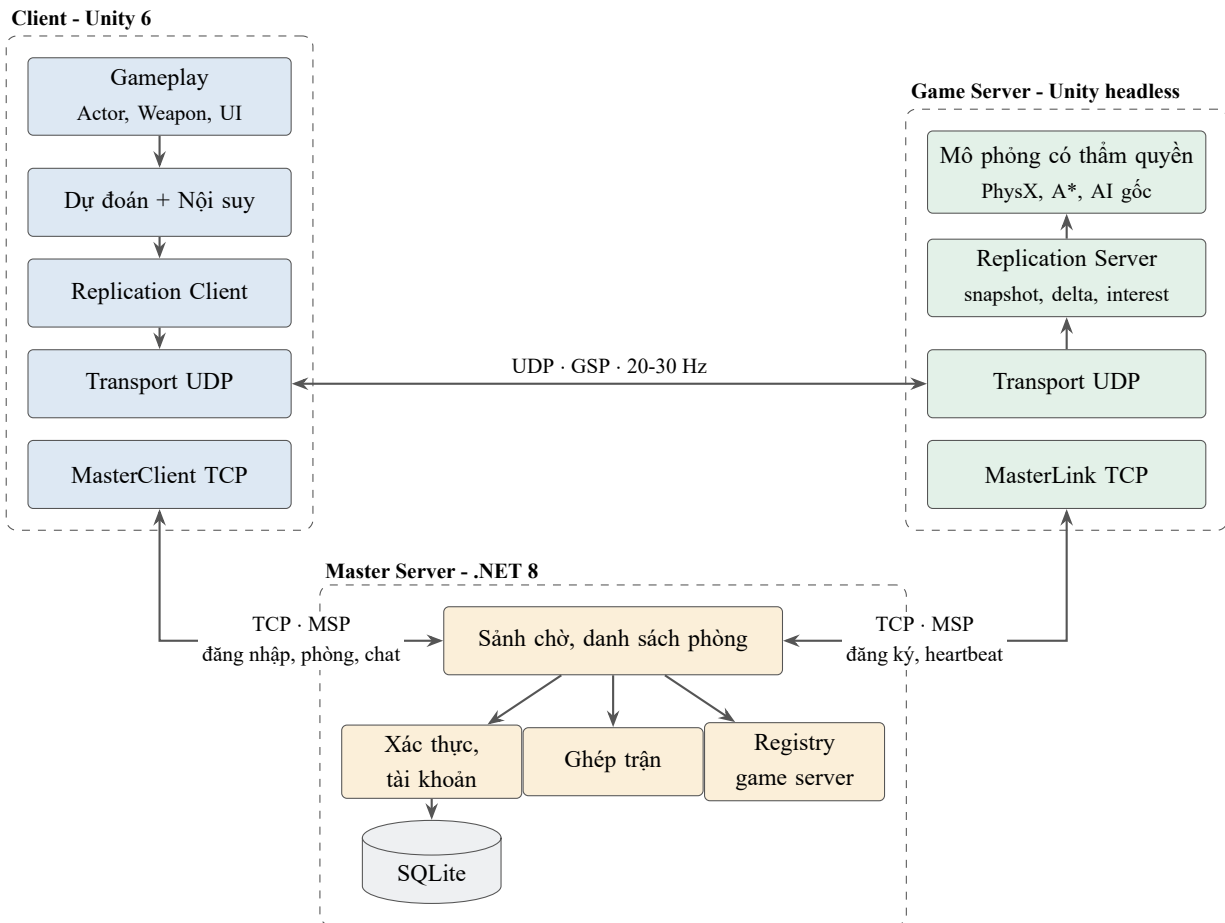
1.1. Bài toán và phạm vi

Ironfront Reborn chuyển mã nguồn trò chơi bắn súng góc nhìn thứ nhất *Ravenfield Beta 5* (Unity 6000.3, vốn chỉ chơi đơn) thành một trò chơi **hiều người chơi, máy chủ nắm quyền quyết định** (server-authoritative). Yêu cầu cốt lõi của môn Lập trình mạng: **toàn bộ tầng mạng được viết tay trên socket TCP và UDP thô** của hệ điều hành - không dùng Mirror, Photon, Netcode for GameObjects, KCP, gRPC hay WebSocket.

Tiêu chí	Giá trị thiết kế
Quy mô một trận	16 người chơi thật + 32 bot AI (tối đa 64 actor, 16 phương tiện)
Tần số mô phỏng / snapshot / gửi input	30 Hz / 20 Hz / 30 Hz
Băng thông xuống mỗi client	≤ 8 KB/s (đo trên server đã triển khai: 1,67 KB/s)
Triển khai	LAN trước, sau đó VPS công cộng (Docker Compose, TLS cho kênh TCP)
Công nghệ	.NET 8 (master server, công cụ), netstandard2.1 (thư viện dùng chung), Unity 6 (client và game server headless)

1.2. Kiến trúc ba tiến trình

Hệ thống gồm **ba loại tiến trình** độc lập, giao tiếp qua **hai giao thức tự định nghĩa**: **MSP** (Master Server Protocol, trên TCP) cho dịch vụ săn chờ và **GSP** (Game Server Protocol, trên UDP) cho lưu lượng trong trận. Hai giao thức gặp nhau ở đúng một điểm: **joinTicket** được master ký HMAC và game server tự kiểm tra.



Hình 1.1. Kiến trúc tổng thể ba tiến trình và hai giao thức MSP/GSP

Tiến trình	Nền tảng	Trách nhiệm chính	Giao tiếp
Client	Unity 6, bản build thường	Hiển thị, nhận input, dự đoán chuyển động người chơi cục bộ, nội suy actor ở xa, giao diện sảnh và HUD	TCP tới Master; UDP tới Game Server
Game Server	Unity 6 headless (chế độ không đồ họa)	Chạy mô phỏng có thẩm quyền (vật lý, AI, bắn, điểm chiếm), tạo snapshot riêng cho từng client, bù trễ	Nhận UDP từ client; TCP tới Master
Master Server	.NET 8 console, SQLite	Tài khoản, phiên, phòng, ghép trận, chat sảnh, registry game server, cấp joinTicket	Nhận TCP từ client và game server

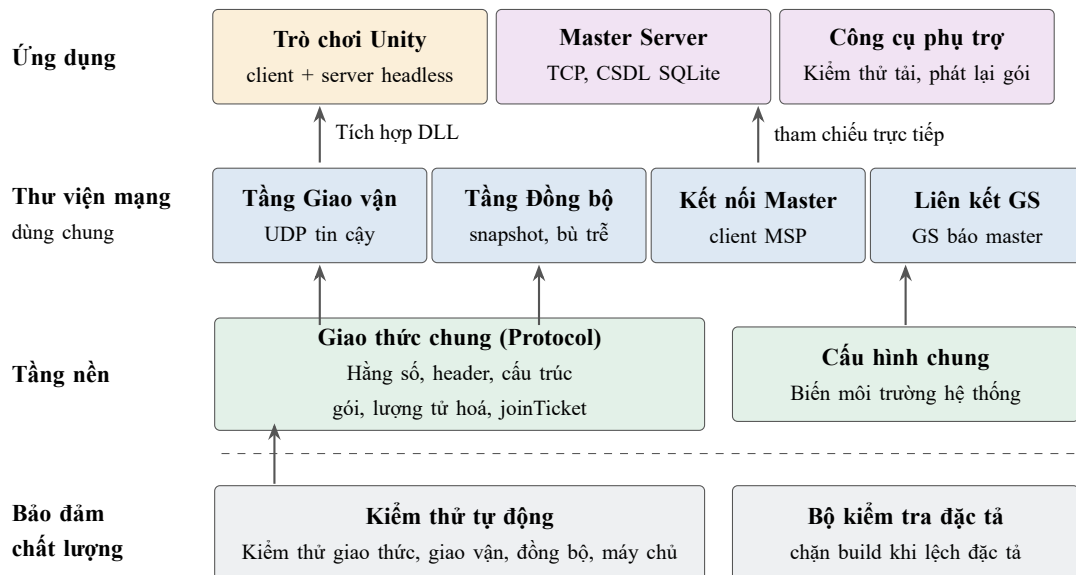
1.3. Phân chia TCP và UDP theo đặc tính dữ liệu

Đặc tính	TCP - Master Server	UDP - Game Server
Dùng cho	Đăng nhập, danh sách phòng, tạo/vào phòng, ghép trận, chat sảnh, heartbeat server	Mọi lưu lượng trong trận: input, snapshot, sự kiện gameplay
Tần suất	Vài gói/phút mỗi client	20-60 gói/giây mỗi chiều
Mất gói	Không chấp nhận (mất LOGIN_RES là treo màn hình)	Chấp nhận với trạng thái: snapshot sau thay thế snapshot trước
Nếu đổi chỗ	Phải tự viết lại độ tin cậy mà TCP đã làm tốt	Head-of-line blocking: một gói mất chặn mọi gói sau; Nagle + delayed ACK thêm 40-200 ms
Đóng khung	Tiền tố độ dài 4 byte + JSON	Datagram tự nhiên, header nhị phân 16 byte

Không dùng WebSocket: WebSocket chạy trên TCP nên thừa hưởng nguyên vẹn head-of-line blocking, cộng thêm chi phí bắt tay HTTP - một ràng buộc chỉ cần cho trình duyệt, không cần cho client desktop.

1.4. Kiến trúc phân lớp hệ thống

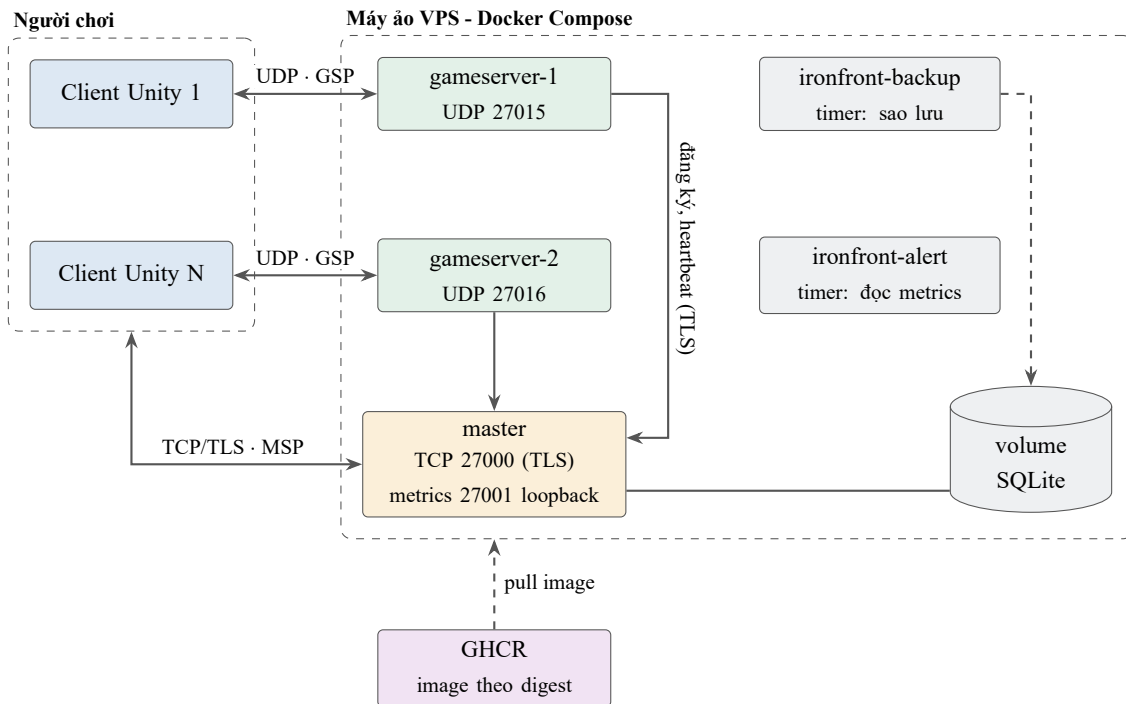
Mã nguồn chia thành các thư viện dùng chung độc lập hoàn toàn với engine đồ hoạ; dự án Unity tích hợp chúng dưới dạng các thư viện liên kết động (DLL). Nhờ vậy logic mạng được kiểm thử tự động độc lập nhanh chóng mà không cần mở Unity Editor.



Hình 1.2. Phân lớp kiến trúc và quan hệ phụ thuộc giữa các phân hệ

Phân hệ / Thành phần	Loại	Quy mô	Vai trò nghiệp vụ
Giao thức chung (Protocol)	Thư viện dùng chung	5 300 dòng	Định dạng gói tin, chuẩn tuần tự hoá và hằng số giao thức
Tầng Giao vận (Transport)	Thư viện giao vận	4 100 dòng	Độ tin cậy trên UDP, kênh truyền, phân mảnh, kiểm soát nghẽn
Đồng bộ và Mô phỏng	Thư viện logic	18 500 dòng	Snapshot, mã hóa delta, vùng quan tâm, tick server, bù trễ
Máy chủ trung tâm (Master)	Ứng dụng dịch vụ	4 300 dòng	Tiếp nhận TCP, xác thực, quản lý phòng, ghép trận, bảo mật
Liên kết máy chủ trung tâm	Thư viện kết nối	1 250 dòng	API trao đổi thông tin với máy chủ trung tâm
Tích hợp giao diện client	Tích hợp ứng dụng	14 600 dòng	Điều phối luồng, dự đoán, nội suy, giao diện sảnh và HUD

1.5. Kiến trúc triển khai



Hình 1.3. Sơ đồ triển khai trên một máy ảo: một master và hai game server headless

Image được triển khai theo bản dựng cố định; khóa bí mật dùng chung và chứng chỉ TLS được cấu hình qua biến môi trường bảo mật độc lập. Sao lưu và cảnh báo chạy bằng bộ định thời hệ thống *ngoài* container, vì bộ phát hiện sự cố phải nằm ngoài thứ nó giám sát. Hạ tầng còn có cấu hình tự động hoá máy ảo và phương án triển khai đám mây thay thế.

1.6. Phân công theo phân hệ

Sinh viên	Mã SV	Phân hệ	Phạm vi nhiệm vụ chính
Nguyễn Thành Trung	B23DCCN861	Client Unity	Giao diện sảnh, HUD, dự đoán và nội suy chuyển động client
Nguyễn Minh Toàn	B23DCCN832	Tầng giao vận UDP	Giao thức UDP tin cậy, kênh truyền, phân mảnh, ACK bitfield
Nguyễn Duy Nghĩa	B23DCCN600	Replication, mô phỏng server	Vòng tick mô phỏng, đồng bộ snapshot, nén delta và bù trễ
Nguyễn Tự Kiên	B23DCCN465	Master Server TCP, hạ tầng	Máy chủ TCP, xác thực, quản lý phòng, ghép trận và hạ tầng VPS

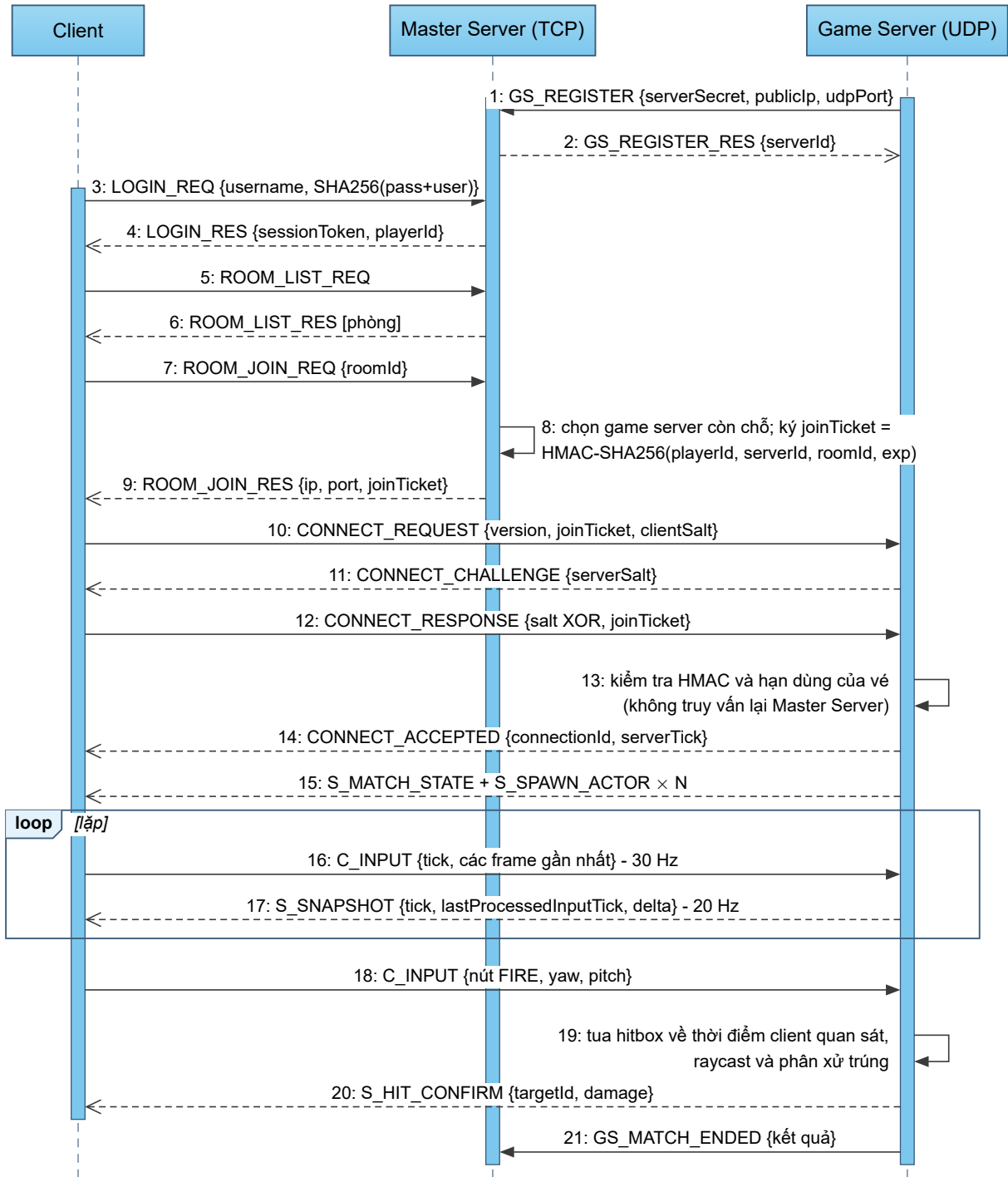
2. Mô tả thiết kế chung của hệ thống

2.1. Mô hình thẩm quyền

Hệ thống **hoàn toàn do server quyết định**: client chỉ gửi *ý định* (input), không bao giờ gửi kết quả. Nhờ vậy gian lận tốc độ, dịch chuyển tức thời, bắn nhanh hay đạn vô hạn trở nên bất khả thi về cấu trúc thay vì chỉ bị phát hiện.

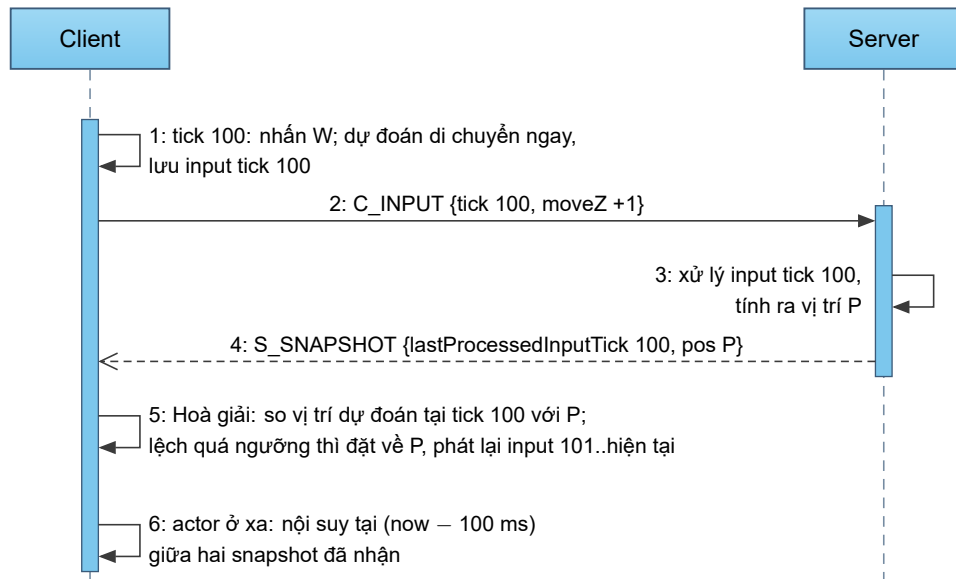
Vấn đề	AI quyết định	Client được làm gì
Vị trí nhân vật	Server	Dự đoán ngay, sau đó hoà giải với server
Trúng đạn, sát thương	Server	Hiện hiệu ứng dự đoán, chờ S_HIT_CONFIRM
Máu, chết, hồi sinh	Server	Chỉ hiển thị
Độ tàn đạn, giật súng	Server	Giật cục bộ cho cảm giác, không ảnh hưởng viên đạn thật
Bot AI, điểm chiếm, điểm số	Server	Nội suy, hiển thị
Lên xuống ghế phương tiện	Server	Gửi yêu cầu C_SEAT_REQUEST
Camera, UI, âm thanh, ragdoll	Client	Toàn quyền, không đồng bộ

2.2. Luồng nghiệp vụ đầu - cuối



Hình 2.1. Luồng đầu - cuối: đăng nhập, vào phòng, kết nối game server; chơi, kết thúc trận

2.3. Các kỹ thuật netcode cốt lõi

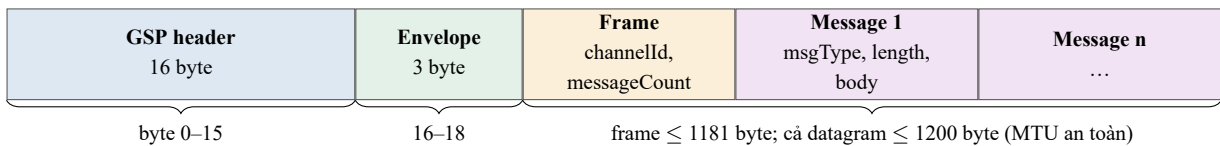


Hình 2.2. Dự đoán phía client, hoà giải với server và nội suy actor ở xa

Kỹ thuật	Mục đích	Tham số
Dự đoán phía client	Người chơi cục bộ di chuyển ngay khi nhấn phím	Cùng thuật toán mô phỏng chuyển động ở client và server
Hoà giải	Sửa sai khi dự đoán lệch, phát lại input chưa xác nhận	Ngưỡng lệch 0,25 m
Nội suy thực thể	Actor ở xa chuyển động mượt dù snapshot chỉ 20 Hz	Bộ đệm 100 ms
Bù trễ	Người ping cao vẫn bắn trúng thứ họ thấy	Tua tối đa 200 ms, lịch sử hitbox 1 s

2.4. Thiết kế giao thức

2.4.1. GSP - cấu trúc một datagram PAYLOAD



Hình 2.3. Ba lớp của một datagram PAYLOAD: header GSP, envelope kênh của transport, frame message của ứng dụng

Kênh	Kiểu	Dữ liệu	Khi mất gói
0	Không tin cậy, không thứ tự	Ping/pong	Bỏ qua
1	Không tin cậy, có thứ tự (server → client)	S_SNAPSHOT, S_WEAPON_FIRE, S_VEHICLE_SNAPSHOT	Bỏ qua; gói cũ bị hủy
2	Tin cậy, có thứ tự	Sự kiện gameplay, spawn/despawn, chat, C_ACK_BASELINE	Gửi lại tới khi được ack
3	Không tin cậy, có thứ tự (client → server)	C_INPUT, C_VEHICLE_INPUT	Bỏ qua; bù bằng dư thừa

Độ tin cậy áp dụng **theo kênh**, không theo kết nối: một sự kiện bị mất trên kênh 2 không làm nguyền snapshot trên kênh 1 - ưu điểm cốt lõi so với TCP vốn ép mọi thứ vào một luồng.

2.4.2. MSP - khung tin trên TCP

length u32, big-endian	msgType u16, little-endian	body UTF-8 JSON, tối đa 64 KB
----------------------------------	--------------------------------------	---

Hình 2.4. Khung tin MSP: tiền tố độ dài giải quyết bài toán ranh giới thông điệp trên luồng byte TCP

2.4.3. joinTicket - cầu nối TCP và UDP

playerId u32	serverId u16	roomId u16	expiresAtUnixMs u64 (60 s)	displayName 16 byte UTF-8	hmac HMAC-SHA256, 32 byte
32 byte dữ liệu được ký					32 byte chữ ký

Hình 2.5. Cấu trúc joinTicket 64 byte

Master ký vé bằng khoá bí mật; game server kiểm tra chữ ký bằng thuật toán so sánh thời gian hằng số và kiểm hạn dùng. Không cần gọi lại master, nên **master có thể dừng giữa trận mà trận vẫn tiếp tục**.

2.5. Tham số thời gian

Tham số	Giá trị	Ghi chú
Tick mô phỏng server	30 Hz (33,3 ms)	Theo đồng hồ thực, không theo chu kỳ cập nhật vật lý
Snapshot	20 Hz (50 ms)	Rẻ hơn 30 Hz, nội suy bù độ mượt
Gửi input	30 Hz	Mỗi gói lặp lại các frame gần nhất
Bộ đệm nội suy	100 ms	Chịu được mất một snapshot
Cửa sổ bù trễ / lịch sử hitbox	200 ms / 1 s	Chặn lạm dụng ping giả
Keep-alive / timeout UDP	1 s / 10 s	
Heartbeat MSP client / game server	15 s	Phát hiện kết nối TCP nửa mở

2.6. Phân loại đối tượng đồng bộ

Loại	Ví dụ	Cách xử lý
Actor được đồng bộ	Người chơi, bot	Có actorId u16, có trong snapshot, mã hoá delta
Phương tiện	Xe, trực thăng, tàu	Luồng riêng S_VEHICLE_SNAPSHOT
Chỉ ở server	Lịch sử hitbox, dữ liệu AI	Không bao giờ gửi
Chỉ ở client	Camera, UI, particle, ragdoll, âm thanh	Không bao giờ gửi
Sự kiện một lần	Chết, nổ, phát súng, chiếm điểm	Kênh tin cậy có thứ tự
Tĩnh	Địa hình, công trình, điểm hồi sinh	Có sẵn trong scene hai phía

2.7. Bảo mật ở mức phù hợp phạm vi

Mối đe dọa	Biện pháp	Tầng
Tăng tốc, dịch chuyển, bắn nhanh, đạn vô hạn	Server chỉ nhận input; kẹp tốc độ; tự quản lý cooldown và đạn	Replication
Mạo danh người chơi	connectionId gắn với playerId trong joinTicket đã ký	Transport
Khuếch đại giả mạo IP	Bắt tay challenge không lưu trạng thái (cookie HMAC)	Transport
Gói rác, lụt gói	Kiểm protocolId; giới hạn tốc độ theo IP; giới hạn nhóm phân mảnh	Transport
Nghe lén mật khẩu	TLS và ghim vân tay chứng chỉ; SHA-256 phía client, bcrypt phía server	Master
Tràn bộ nhớ, Slowloris	Khung tối đa 64 KB; hạn 30 s trước xác thực; 5 kết nối/IP	Master
SQL injection, dò mật khẩu	Truy vấn tham số hoá; 5 lần/phút/IP; khoá 15 phút sau 10 lần sai	Master

2.8. Các quyết định kiến trúc đã chốt

Mã	Quyết định	Lý do
AD-1	Server nắm quyền, không có listen-server	Thảm quyền rõ ràng; tránh sửa 21 singleton
AD-2	Game server là Unity headless	Tái dùng PhysX, A*, AI; tiết kiệm 8-12 tuần
AD-3	Không theo đuổi tắt định (lockstep)	Bộ sinh ngẫu nhiên rải rác; engine vật lý không tắt định
AD-4	Ragdoll chỉ là hiệu ứng cục bộ	Tiết kiệm khoảng 1,7 MB/s
AD-5	Transport, Replication là thư viện .NET thuần	Kiểm thử bằng xUnit, tránh xung đột Unity
AD-7	Snapshot không tin cậy, sự kiện tin cậy	Đúng bản chất trạng thái và sự kiện
AD-8	Không dùng WebSocket	Yêu cầu môn học và head-of-line blocking

2.9. Quy trình phát triển và bảo đảm chất lượng

- **Đặc tả giao thức đóng băng** là nguồn sự thật duy nhất của hệ thống; công cụ kiểm tra tự động sẽ cảnh báo nếu cấu trúc gói tin của các bên không đồng nhất.
- **Tách người thiết kế và người kiểm thử:** nhóm giao vận xây dựng cơ chế đóng gói dữ liệu, nhóm đồng bộ xây dựng bộ kiểm thử tính tương thích theo đặc tả.
- **Tích hợp liên tục (CI)** chạy tự động: kiểm tra toàn diện, coi cảnh báo là lỗi, chạy toàn bộ bộ kịch bản kiểm thử.

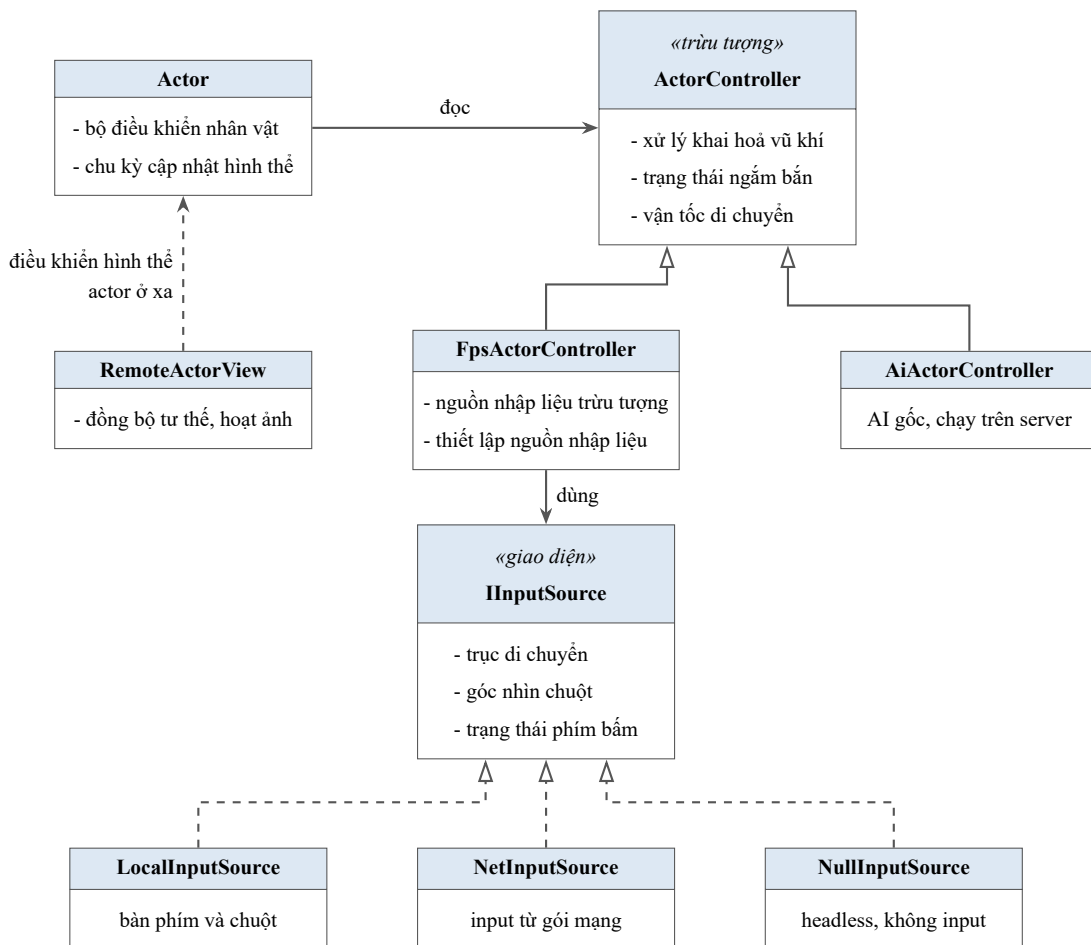
- **Tiêu chuẩn hoàn thành (Definition of Done):** hoàn thành kiểm thử thực tế, toàn bộ kiểm nghiệm tự động đạt chuẩn, ứng dụng biên dịch sạch sẽ, tài liệu kỹ thuật hoàn thiện.

II. NỘI DUNG CÁ NHÂN - Phân hệ client Unity.

3. Biểu đồ và giao diện nội dung cá nhân thực hiện

Phân hệ client là nơi các thư viện mạng được đưa vào trò chơi thật. Nhiệm vụ gồm: mở **điểm nối netcode** trong mã gameplay một người chơi mà không phá lối chơi gốc; làm cho **người chơi ở xa hiển thị mượt** (nội suy, hoạt ảnh, ragdoll cục bộ); làm cho **người chơi cục bộ phản hồi tức thì** (dự đoán và hoà giải); và xây dựng **giao diện** sảnh chờ, HUD, overlay chẩn đoán. Đây cũng là thành viên duy nhất mở Unity Editor.

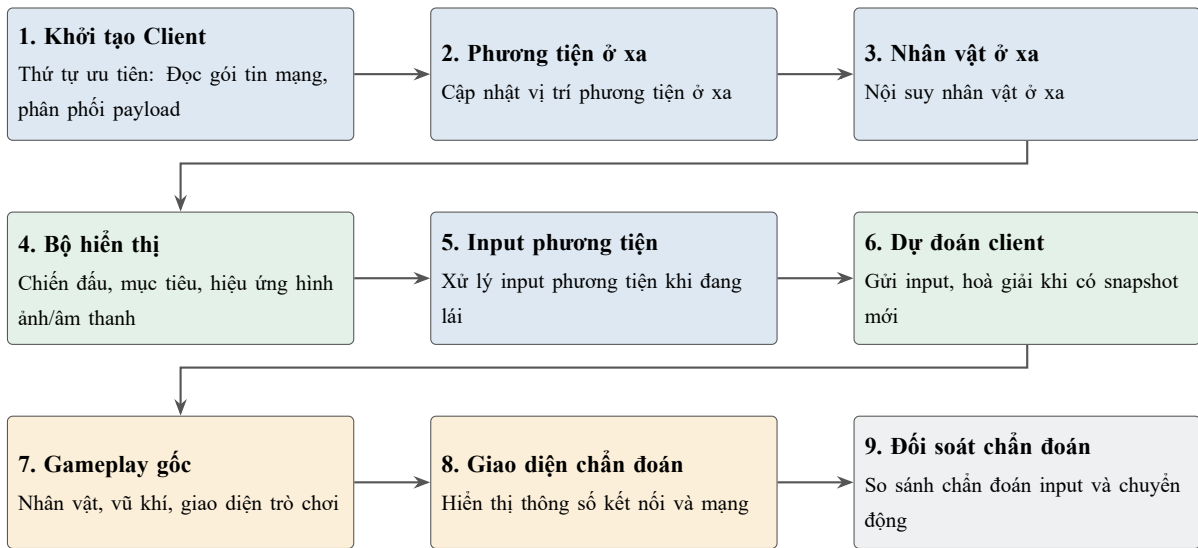
3.1. Điểm nối netcode trong mã gameplay



Hình 3.1. Biểu đồ cấu trúc lớp điểm nối: trừu tượng hoá nguồn điều khiển nhân vật

Thành phần quản lý nhân vật không phụ thuộc vào nguồn gốc dữ liệu đầu vào. Việc trừu tượng hóa các thao tác điều khiển ra sau giao diện nguồn nhập liệu giúp cùng một bộ điều khiển có thể nhận tín hiệu từ bàn phím (ở client), từ gói tin mạng (ở server) hoặc chế độ tự động không người lái (headless).

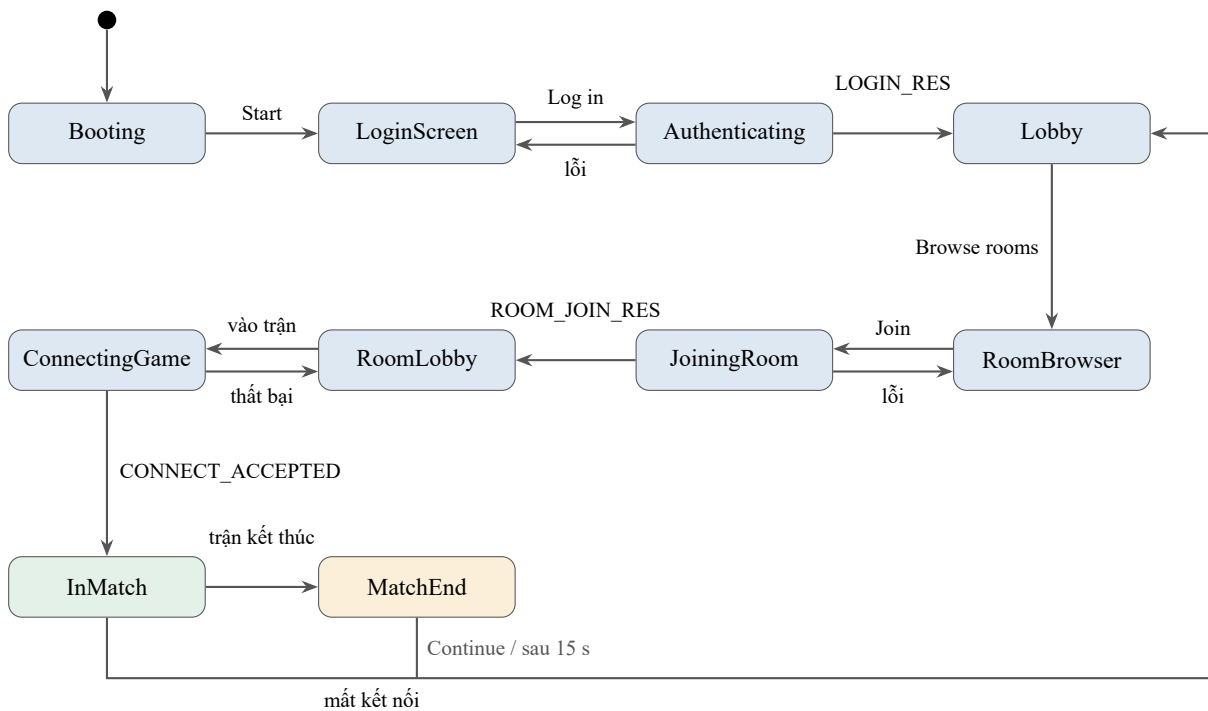
3.2. Thứ tự thực thi một khung hình client



Hình 3.2. Chuỗi thứ tự thực thi thành phần client trong một khung hình

Mọi dữ liệu đến được áp dụng **trước** khi có thành phần nào đọc nó: thành phần giao vận chạy sớm nhất để xác định vai trò mạng trước khi các đối tượng khởi tạo, và các bước chẩn đoán chạy sau cùng.

3.3. Máy trạng thái luồng trò chơi



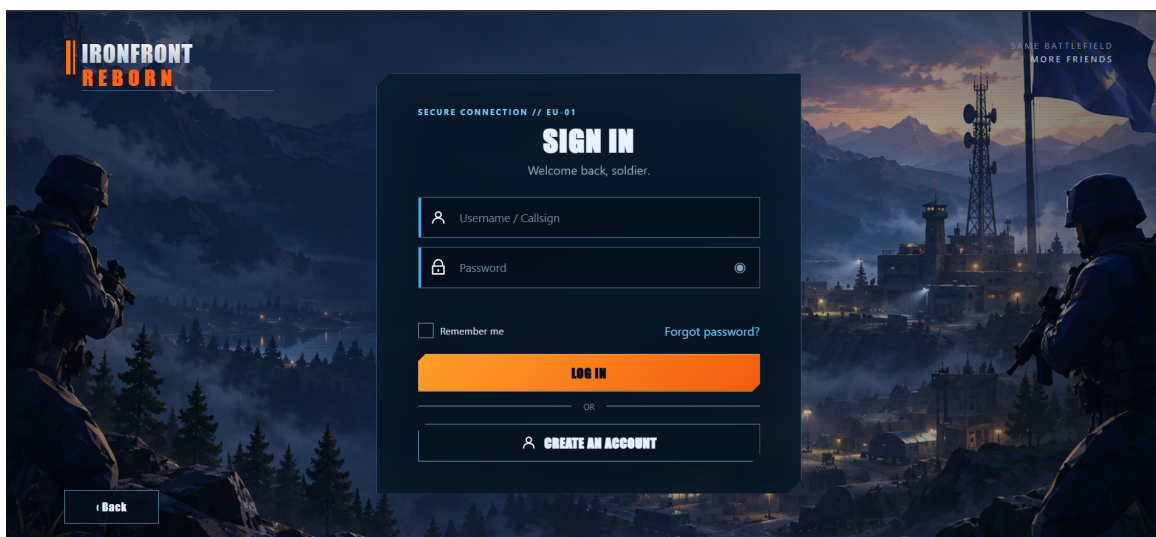
Hình 3.3. Máy trạng thái GameFlowController: 10 trạng thái; mọi chuyển trạng thái ngoài bảng đều bị từ chối

3.4. Giao diện sảnh chờ

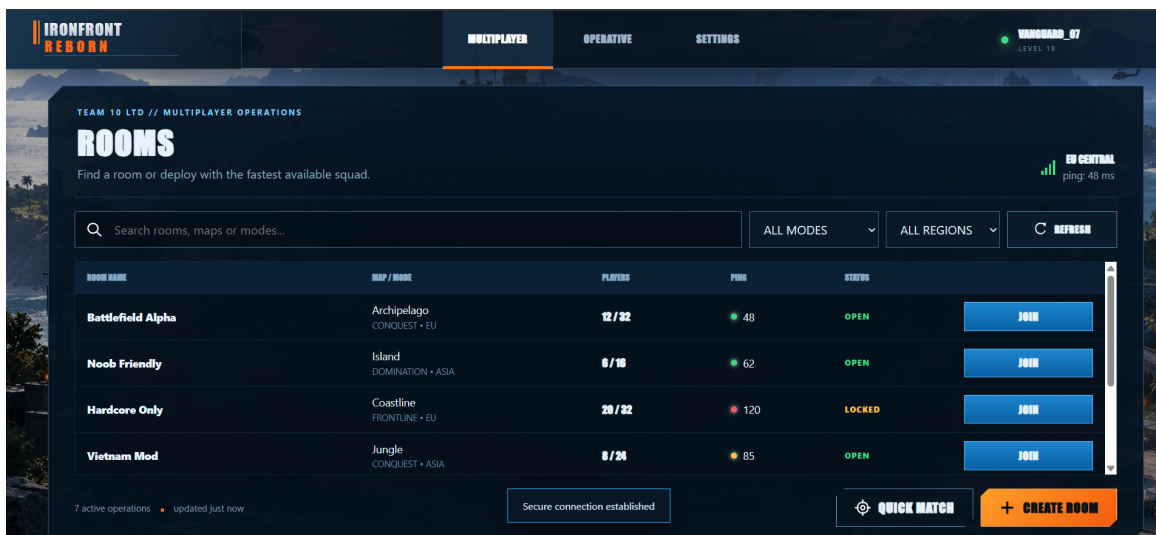
Người chơi đi qua các màn hình theo đúng thứ tự các trạng thái ở Hình 3.3: màn hình chính, đăng nhập, rồi danh sách phòng. Ngoài luồng chơi mạng còn có màn hình luyện tập ngoại tuyến và màn hình cài đặt. Giao diện chẩn đoán sảnh chờ được tích hợp làm công cụ gỡ lỗi nhanh, kích hoạt qua phím tắt Shift+F2.



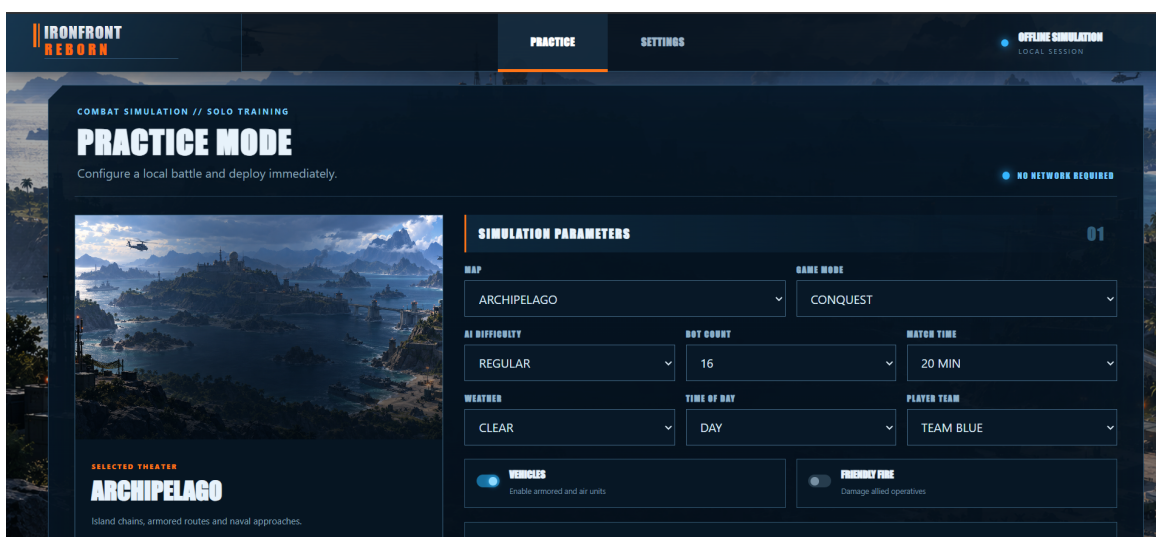
Hình 3.4. Màn hình chính: chơi mạng (Multiplayer), luyện tập ngoại tuyến, cài đặt và thoát



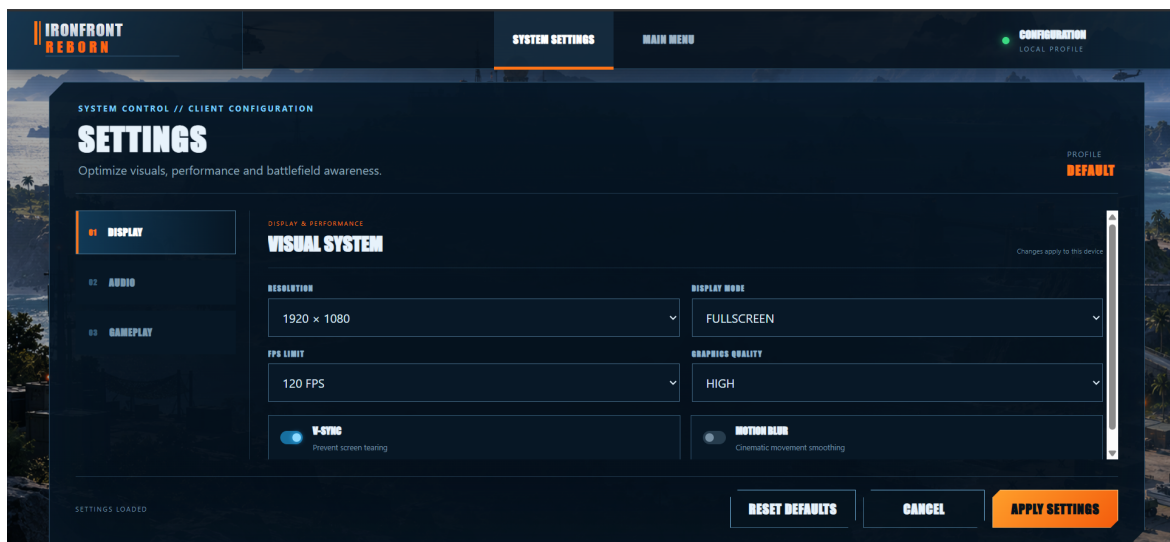
Hình 3.5. Màn hình đăng nhập (trạng thái LoginScreen / Authenticating): tên đăng nhập, mật khẩu, tạo tài khoản mới



Hình 3.6. Danh sách phòng (trạng thái RoomBrowser): tìm kiếm, lọc theo chế độ và khu vực, làm mới, vào phòng, ghép trận nhanh và tạo phòng

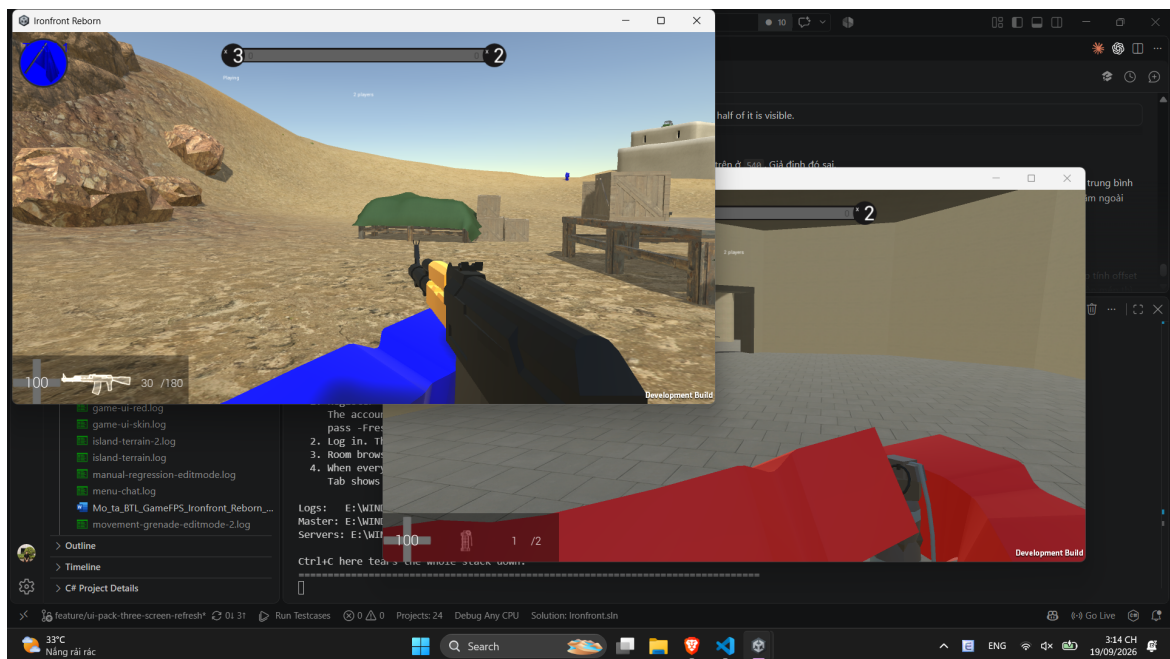


Hình 3.7. Màn hình luyện tập ngoại tuyến: chọn bản đồ, chế độ chơi, độ khó và số lượng bot, không cần kết nối mạng



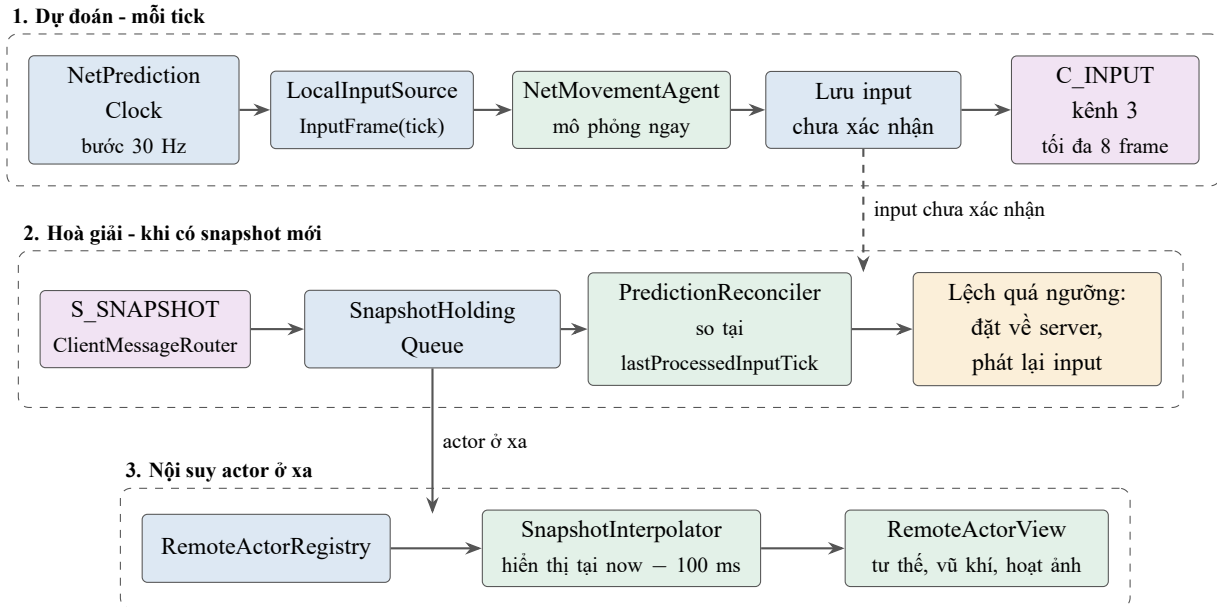
Hình 3.8. Màn hình cài đặt: hiển thị, âm thanh và lối chơi, lưu theo máy

3.5. Giao diện trong trận và overlay chẩn đoán



Hình 3.9. Màn hình trong trận đấu

3.6. Luồng dự đoán và hoà giải phía client



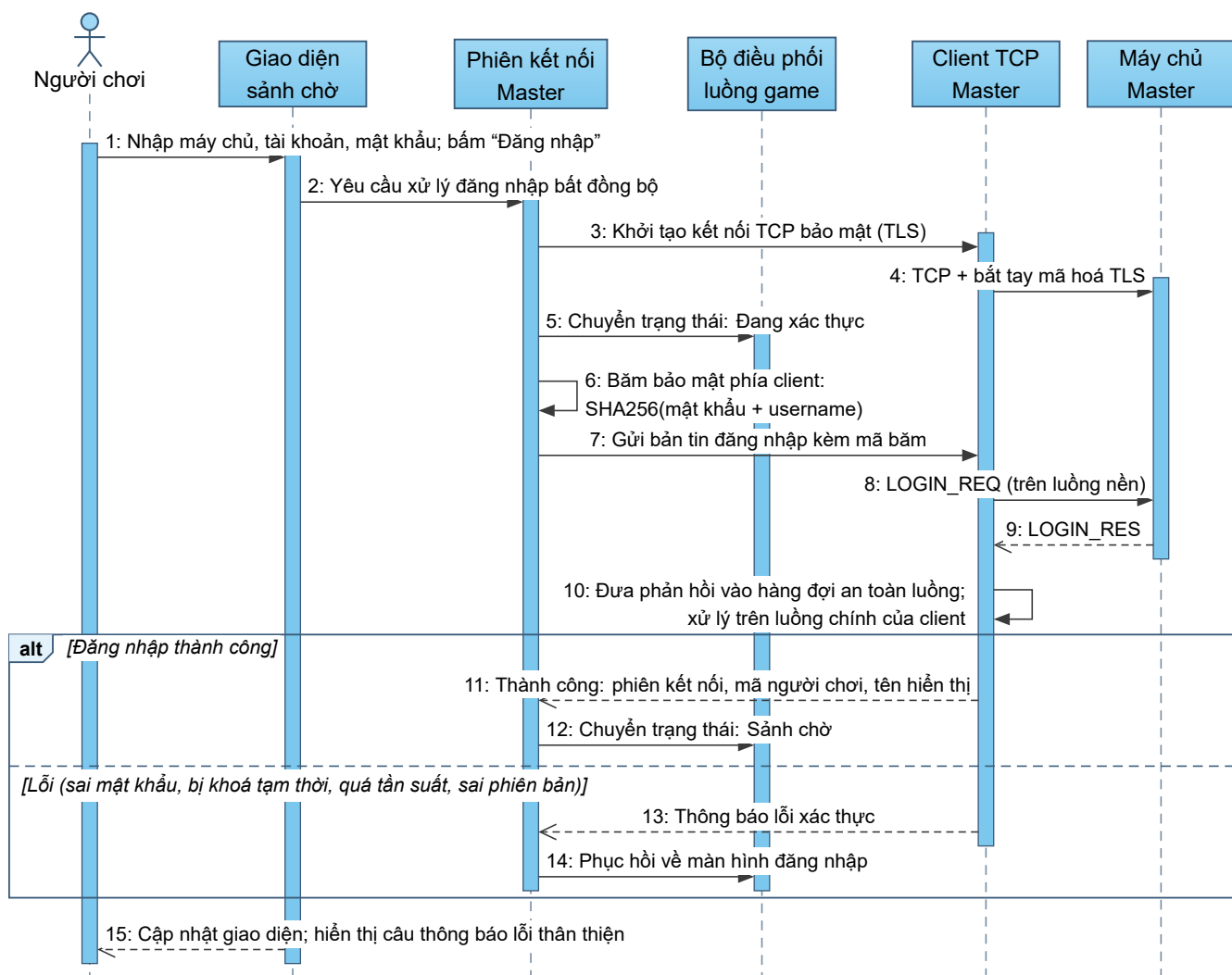
Hình 3.10. Luồng xử lý dự đoán, hoà giải và nội suy trong client

4. Luồng xử lý và giải pháp đã ứng dụng

Mục này mô tả chi tiết các luồng mà phân hệ client hiện thực: từ màn hình đăng nhập tới sảnh, vào phòng và kết nối game server bằng vé, rồi trong trận là dự đoán - hoà giải chuyển động của người chơi cục bộ, nội suy người chơi ở xa, hiển thị sự kiện chiến đấu và HUD, cuối cùng là mất kết nối và kết thúc trận.

4.1. Luồng đăng nhập từ giao diện

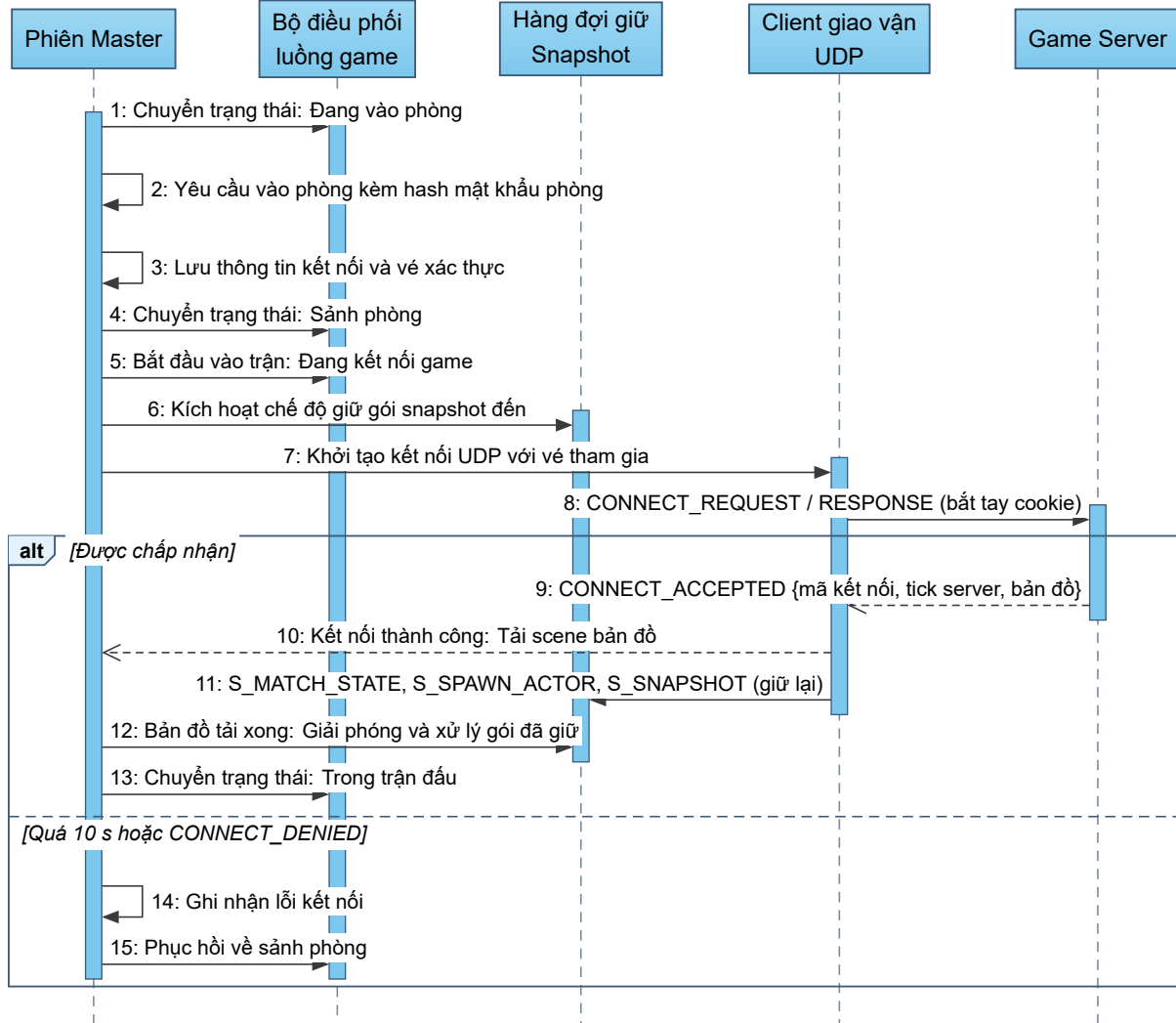
Bài toán. Luồng dựng giao diện chạy trên luồng chính của client; nếu dùng chờ phản hồi mạng đồng bộ thì giao diện sẽ bị treo đóng băng. Ngược lại, các sự kiện nhận dữ liệu mạng chạy từ luồng nền không được phép trực tiếp thao tác lên cây đối tượng hiển thị.



Hình 4.1. Biểu đồ tuần tự đăng nhập: giao diện không bao giờ chặn, mọi kết quả về luồng chính an toàn

Vấn đề	Giải pháp đã áp dụng
Giao diện không thể dừng chờ mạng	Xử lý mạng bất đồng bộ có khối bắt lỗi bao trọn, bảo đảm ngoại lệ không thoát ra làm gián đoạn luồng chính
Sự kiện mạng chạy trên luồng nền	Sử dụng hàng đợi an toàn luồng để chuyển dữ liệu từ luồng nhận sang xử lý tại luồng giao diện ở mỗi khung hình
Chuyển màn hình sai thứ tự khi thao tác chồng nhau	Quản lý bằng máy trạng thái 10 trạng thái có bảng chuyển dịch hợp lệ; mọi thao tác sai luồng đều bị từ chối và kích hoạt quy trình phục hồi
Mật khẩu lộ ra đường truyền hoặc log máy chủ	Bấm SHA-256 kết hợp tên người dùng ngay tại client trước khi truyền; mật khẩu phòng cũng được băm độc lập
Mã lỗi kỹ thuật khó hiểu	Bảng ánh xạ mã lỗi hệ thống sang thông điệp tiếng Việt thân thiện, rõ ràng cho người chơi

4.2. Luồng vào phòng và kết nối game server

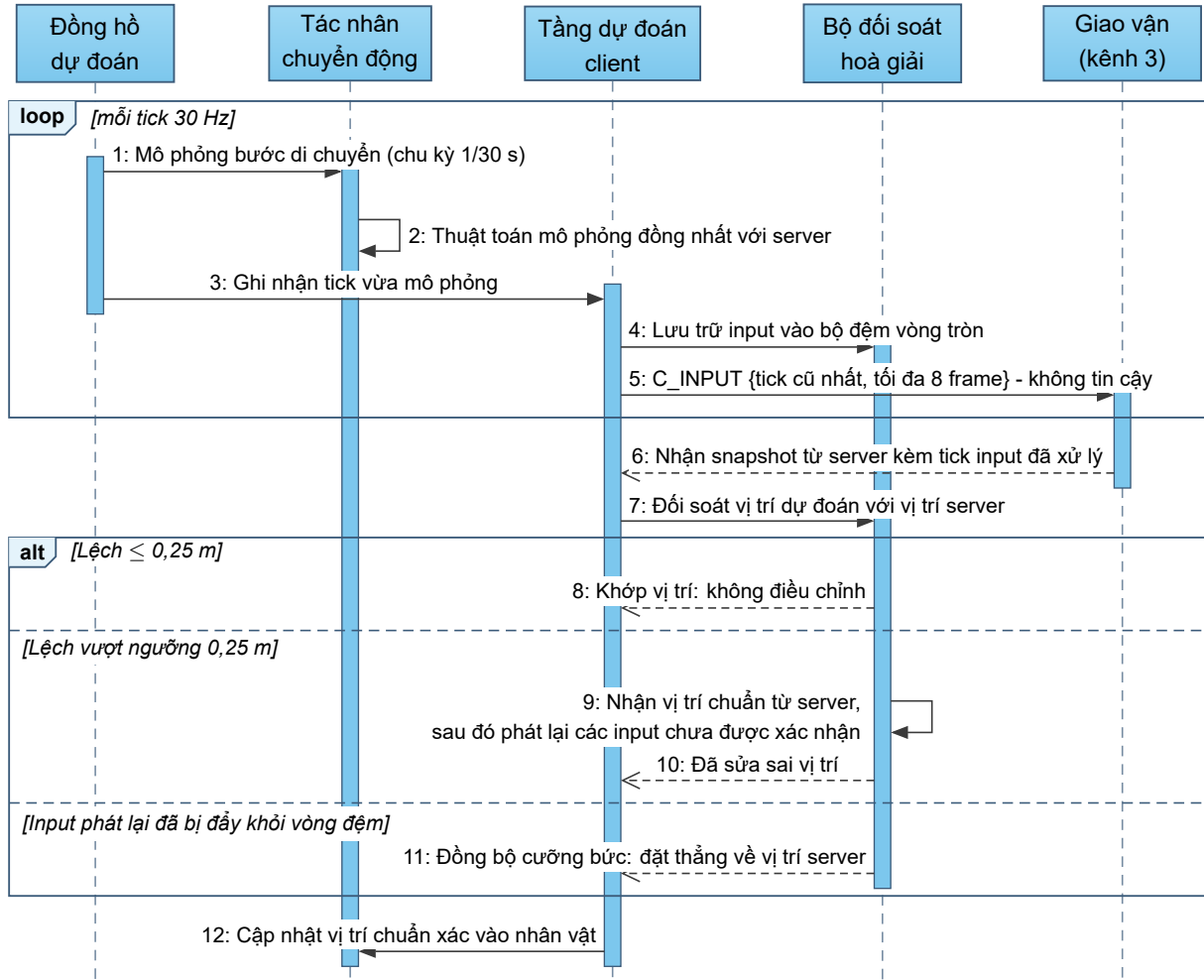


Hình 4.2. Biểu đồ tuần tự từ vào phòng tới vào trận: vé từ master, bắt tay UDP, giữ gói trong lúc tải bản đồ

Các giải pháp kỹ thuật chính.

- **Gói trọn địa chỉ, cổng và vé vào một cấu trúc dữ liệu duy nhất.** Tránh việc gọi nhầm server này với vé của server khác (game server sẽ từ chối với lỗi chữ ký không nói rõ nguyên nhân). Vé chỉ sống 60 s nên client kết nối ngay và hết hạn chờ sau 10 s - nằm gọn trong hạn của vé.
- **Giữ gói trong lúc tải bản đồ.** Server bắt đầu gửi trạng thái ngay sau khi chấp nhận kết nối, khi bản đồ còn đang tải. Hàng đợi giữ snapshot bắt đầu kích hoạt **trước** khi gửi gói kết nối, và chỉ phát lại khi bản đồ sẵn sàng; vì thế không có khoảng hờ nào để một gói bị đưa vào đối tượng chưa tồn tại.
- **Xử lý lỗi kết nối an toàn.** Vé sai định dạng, cổng ngoài phạm vi hay địa chỉ không phân giải được đều được xử lý an toàn thành thông báo và trạng thái tự động quay về sảnh phòng.
- **Kết nối trực tiếp không cần master:** Hỗ trợ chế độ kết nối trực tiếp sử dụng vé mặc định dành riêng cho việc kiểm thử độc lập trong mạng cục bộ.

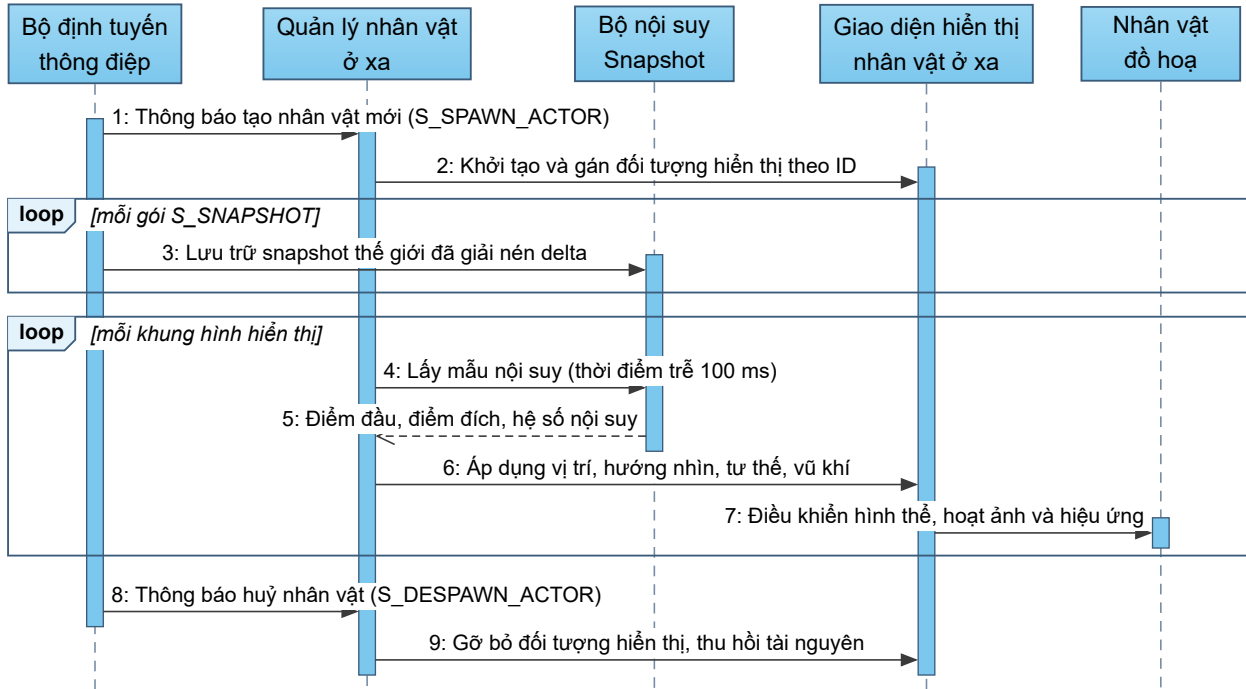
4.3. Luồng dự đoán và hoà giải chuyển động người chơi cục bộ



Hình 4.3. Biểu đồ tuần tự dự đoán phía client và hoà giải với snapshot của server

Vấn đề	Giải pháp đã áp dụng
Chờ server trả lời mới di chuyển thì trễ bằng một vòng RTT	Dự đoán: mô phỏng ngay tại client bằng cùng thuật toán vật lý với server nên cùng input cho cùng kết quả
Gói input bị mất	Mỗi gói mang tối đa 8 frame gần nhất (266 ms dư thừa ở 30 Hz), gửi không tin cậy trên kênh 3 - rẻ hơn trả ack cho 30 bản tin mỗi giây
Dự đoán lệch server	So sánh tại tick input gần nhất server đã xử lý; lệch quá 0,25 m thì nhận trạng thái server trước rồi mới phát lại input chưa xác nhận (phát lại lên trạng thái dự đoán sẽ nhân đôi sai số)
Ghi input sai tick làm mọi lần phát lại lệch khung	Lưu input vào bộ đệm trước khi gửi và dùng đúng số tick mà đồng hồ đã đóng dấu
Số thứ tự tick quay vòng	So sánh bằng số học modulo tuần hoàn; dữ liệu cũ bị loại bỏ
Cập nhật vị trí nhân vật quá nhiều	Chỉ cập nhật lại vị trí nhân vật khi có sự sai lệch vị trí vượt ngưỡng quy định

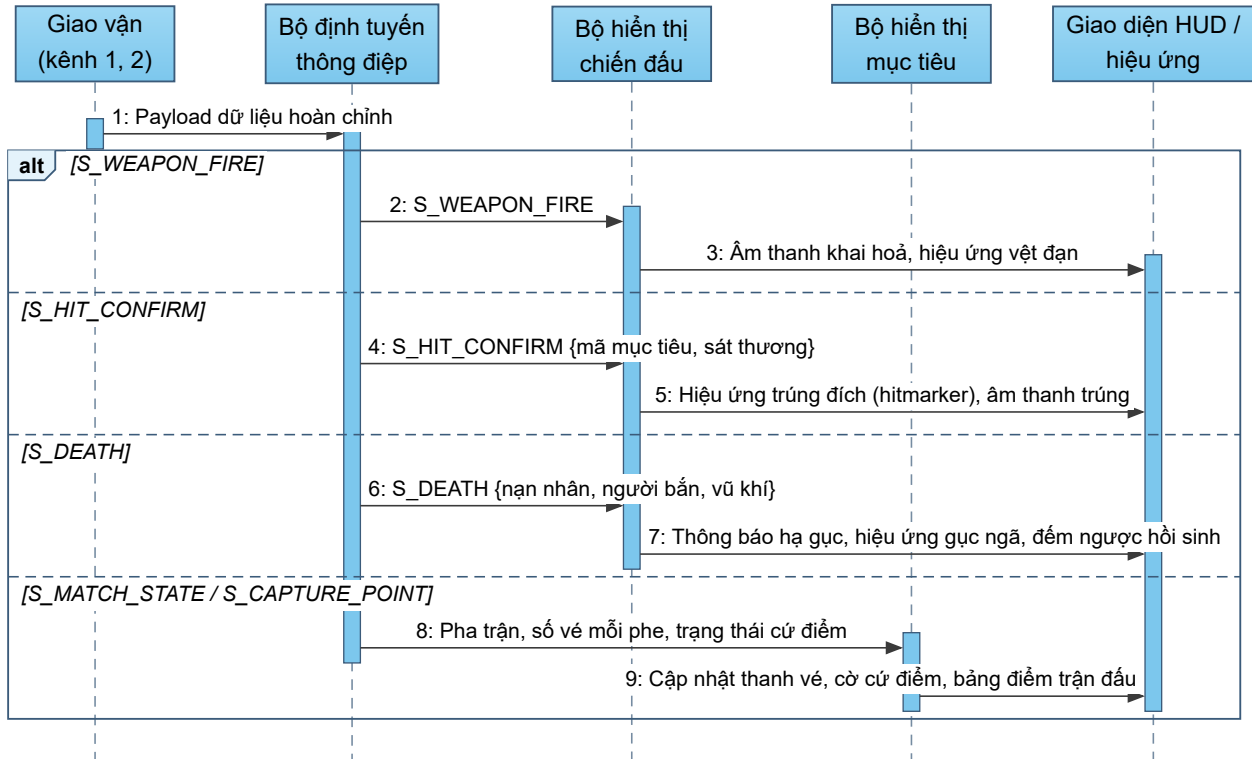
4.4. Luồng hiển thị người chơi và bot ở xa



Hình 4.4. Biểu đồ tuần tự nội suy actor ở xa giữa hai snapshot đã nhận

Giải pháp. Snapshot chỉ tới 20 Hz (xa hơn còn 10 hoặc 4 Hz) nên hiển thị thẳng sẽ giật. Client hiển thị lùi sau snapshot mới nhất 2 tick và **nội suy** giữa hai snapshot bao quanh thời điểm đó - mất một snapshot vẫn còn cặp khác để nội suy, đúng yêu cầu “chuyển động của người khác luôn mượt khi mất gói”. Vòng đệm 16 snapshot (nửa giây) được cấp phát một lần; snapshot được sao chép vào bộ đệm nội suy riêng biệt. Nếu thời điểm hiển thị cũ hơn snapshot cũ nhất thì dừng ở snapshot cũ nhất chứ không ngoại suy lùi. Hiệu ứng gục ngã khi chết chỉ là hiệu ứng cục bộ, không đồng bộ (tiết kiệm băng thông đáng kể).

4.5. Luồng sự kiện chiến đấu, mục tiêu và HUD

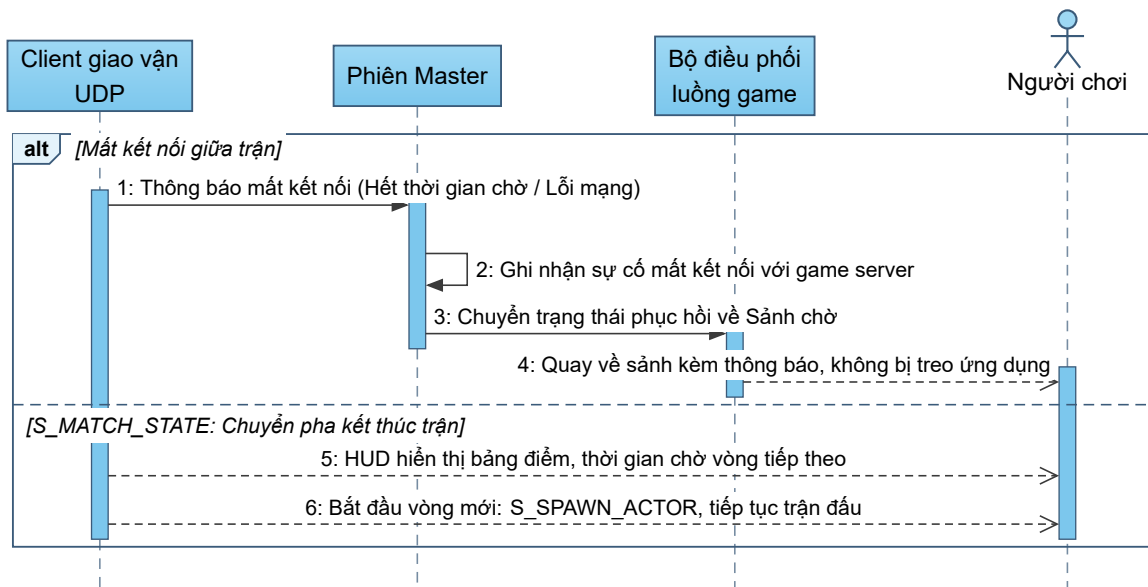


Hình 4.5. Biểu đồ tuần tự biến sự kiện server thành hiệu ứng và HUD

Máu, đạn, hạ gục và điểm số đều **do server quyết định**; client chỉ hiển thị. Giật súng cục bộ vẫn chạy để có cảm giác bắn, nhưng không ảnh hưởng viên đạn thật. Trước khi áp dụng lên đối tượng, mọi bộ hiển thị kiểm tra hai điều kiện: “có phải vai trò client mạng không” và “đối tượng có phải người đang ngồi trước máy không” - để hiệu ứng dành riêng cho người chơi (dấu trúng đạn, đếm ngược hồi sinh) không bị áp nhầm lên nhân vật khác ở xa.

4.6. Luồng mất kết nối và kết thúc trận

Theo đề, hết trận server hiện bảng điểm 20 s rồi đặt lại thế giới, người chơi không phải thoát phòng; còn người chơi mất kết nối quá 10 s bị coi như rời trận. Phía client xử lý hai trường hợp như sau.



Hình 4.6. Biểu đồ tuần tự phía client khi mất kết nối và khi trận kết thúc

Khi server không nhận được gói nào trong 10 s, nó coi người chơi đã rời trận; phía client cũng tự hết hạn sau 10 s im lặng và đưa người chơi về sảnh với thông báo, thay vì để lại một thể giới đứng yên không ai cập nhật.

4.7. Bảng ánh xạ hình vẽ với chức năng cá nhân thực hiện

Bảng tổng hợp đối chiếu toàn bộ các sơ đồ kiến trúc, giao diện và luồng xử lý do cá nhân thực hiện tương ứng với các chức năng cụ thể trong phân hệ client Unity:

Hình vẽ	Tên hình vẽ / biểu đồ	Chức năng cá nhân thực hiện
Hình 3.1	Biểu đồ cấu trúc lớp điểm nối	Trừu tượng hoá và tách biệt nguồn dữ liệu điều khiển nhân vật khỏi logic hiển thị; hỗ trợ đa nguồn input (bàn phím chuột, gói mạng UDP từ server hoặc bot AI).
Hình 3.2	Thứ tự thực thi khung hình	Thiết lập chu trình thực thi các thành phần mạng client theo thứ tự ưu tiên: nhận và giải mã gói tin trước, cập nhật hiển thị sau, tránh hiện tượng lệch frame.
Hình 3.3	Máy trạng thái luồng game	Quản lý máy trạng thái 10 trạng thái toàn cục của client; chặn mọi chuyển đổi trạng thái bất hợp lệ và bảo đảm quy trình phục hồi an toàn khi lỗi mạng.
Hình 3.4	Màn hình chính	Xây dựng giao diện menu chính: điều hướng chế độ chơi mạng (Multiplayer), luyện tập offline, thiết lập cài đặt và thoát trò chơi.
Hình 3.5	Màn hình đăng nhập	Thiết kế biểu mẫu nhập thông tin, xử lý băm mật khẩu bảo mật tại máy khách và hiển thị thông báo lỗi phản hồi từ máy chủ trung tâm.
Hình 3.6	Danh sách phòng chơi	Hiện thực giao diện sảnh chờ: tìm kiếm phòng, lọc phòng theo chế độ/khu vực, hiển thị danh sách phòng trực quan, tạo phòng và ghép trận nhanh.
Hình 3.7	Màn hình luyện tập	Xây dựng tính năng luyện tập ngoại tuyến: chọn bản đồ, tùy chỉnh số lượng bot AI, chế độ chơi mà không cần kết nối mạng.
Hình 3.8	Màn hình cài đặt	Thiết kế hệ thống cài đặt client: cấu hình hiển thị đồ họa, âm lượng và độ nhạy chuột, lưu trữ cấu hình cục bộ theo máy.
Hình 3.9	Giao diện HUD trong trận	Tích hợp giao diện HUD chiến đấu: thanh máu, lượng đạn, la bàn định hướng, tiến độ điểm chiếm, bảng điểm và hiệu ứng phản hồi trúng đạn.
Hình 3.10	Sơ đồ xử lý dự đoán và nội suy	Thiết kế kiến trúc xử lý kép: mô phỏng dự đoán cục bộ phản hồi tức thời kết hợp bộ đệm nội suy mượt mà đối với người chơi ở xa.
Hình 4.1	Sơ đồ tuần tự đăng nhập	Hiện thực luồng đăng nhập bất đồng bộ: không chặn luồng giao diện, băm SHA-256 mật khẩu và chuyển kết quả an toàn về luồng chính.
Hình 4.2	Sơ đồ tuần tự vào trận	Hiện thực luồng vào phòng, nhận vé xác thực 64 byte từ master, bắt tay UDP với game server và lưu đệm gói tin khi đang tải bản đồ.
Hình 4.3	Sơ đồ tuần tự dự đoán	Hiện thực thuật toán dự đoán di chuyển 30 Hz phía client, lưu trữ bộ đệm input và hoà giải sai lệch vị trí khi nhận snapshot chuẩn từ server.
Hình 4.4	Sơ đồ tuần tự nội suy	Hiện thực thuật toán lấy mẫu nội suy giữa hai snapshot lịch sử (lùi 100 ms), đảm bảo hiển thị mượt mà ngay cả khi mất gói mạng.
Hình 4.5	Sơ đồ tuần tự sự kiện	Tiếp nhận và giải mã các sự kiện chiến đấu có thẩm quyền từ server; kích hoạt âm thanh, dấu trúng đạn (hitmarker) và cập nhật thanh điểm.
Hình 4.6	Sơ đồ tuần tự rời trận	Xử lý tự động hết thời gian chờ ngắt kết nối (10 s im lặng), bảo vệ trò chơi không bị treo cứng và quay trở lại sảnh chờ an toàn.